



GRAFCET en lenguaje de contactos (KOP)

Sistemas secuenciales
programables





Índice

7.1. Administración del GRAFCET

- 7.1.1. Zona de inicialización
- 7.1.2. Zona secuencial
- 7.1.3. Zonas de acciones

7.2. Programación de la zona secuencial

- 7.2.1. Direccionamiento de las etapas
- 7.2.2. Segmento modelo de etapa
- 7.2.3. Combinaciones lógicas en transiciones
- 7.2.4. Divergencia de secuencias opcionales (OR)
- 7.2.5. Convergencia de secuencias opcionales (OR)
- 7.2.6. Divergencia de secuencias simultáneas o sincronizadas (AND)
- 7.2.7. Convergencia de secuencias simultáneas o sincronizadas (AND)
- 7.2.8. Temporizaciones en las transiciones
- 7.2.9. Contadores en las transiciones
- 7.2.10. Saltos y retornos desde múltiples etapas

7.3. Programación de la zona de inicialización

- 7.3.1. Activación de etapas iniciales

7.4. Programación de la zona de acciones

- 7.4.1. Asignación directa o acción continua
- 7.4.2. Acción condicionada
- 7.4.3. Asignación a la activación (SET)
- 7.4.4. Asignación a la desactivación (RESET)
- 7.4.5. Acciones condicionadas a temporización
- 7.4.6. Acciones de asignación por flanco
- 7.4.7. Acciones condicionadas a eventos
- 7.4.8. Acciones con contadores
- 7.4.9. Acciones en SPF

7.5. Zona no secuencial



Introducción

El GRAFCET se podrá implementar en cualquier lenguaje de programación, ya sea de tipo general o cualquiera que sea para hacer procesos industriales en PLC.

En este tema se usará el lenguaje de contactos KOP que habrá que implementarlo en los dispositivos de Siemens que usan STEP 7. Todos estos ejemplos serán válidos para todos los modelos de Siemens.

Al finalizar esta unidad

- + Conoceremos las diferentes zonas de un GRAFCET en los bloques de programación
- + Asociaremos algunos elementos a dichos GRAFCET
- + Utilizaremos un sistema metódico para programar
- + Veremos algunos ejemplos y líneas de código

7.1.

Administración del GRAFCET

La programación del GRAFCET la estructuraremos de la siguiente manera:

- > Zona de inicialización
- > Zona secuencial
- > Zona de acciones

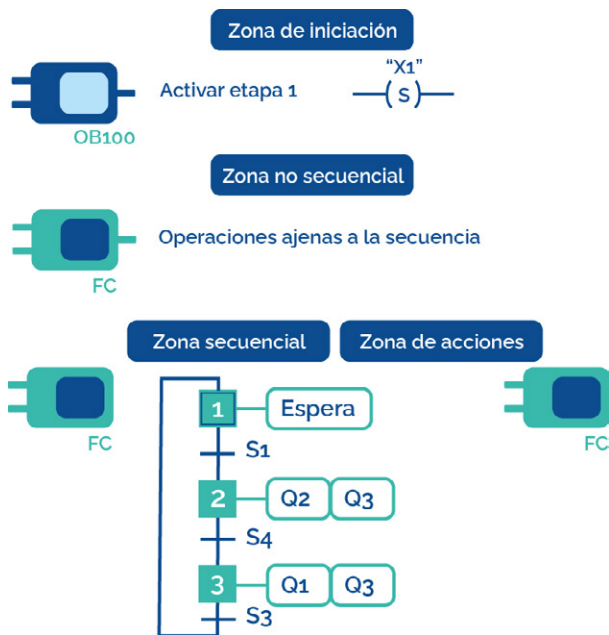


Imagen 1. Zonas para administrar un GRAFCET

7.1.1. Zona de inicialización

Se trata de la zona que representa todo lo que tendrá que ocurrir para que se inicie la ejecución del programa. En esta zona deberemos activar las etapas iniciales del GRAFCET y desactivamos los restantes.

Se inicializarán los valores como contadores, valores de consigna, etc. El programa de la zona preliminar se hará cuando el PLC pase de STOP a RUN.

7.1.2. Zona secuencial

En esta zona se programarán las etapas y transiciones del GRAFCET. La zona secuencial se escribirá como un bloque FC que se le llamará desde el bloque principal OB1. Si disponemos de una secuencia con más de un GRAFCET, los podríamos programar en bloques FC independientes.

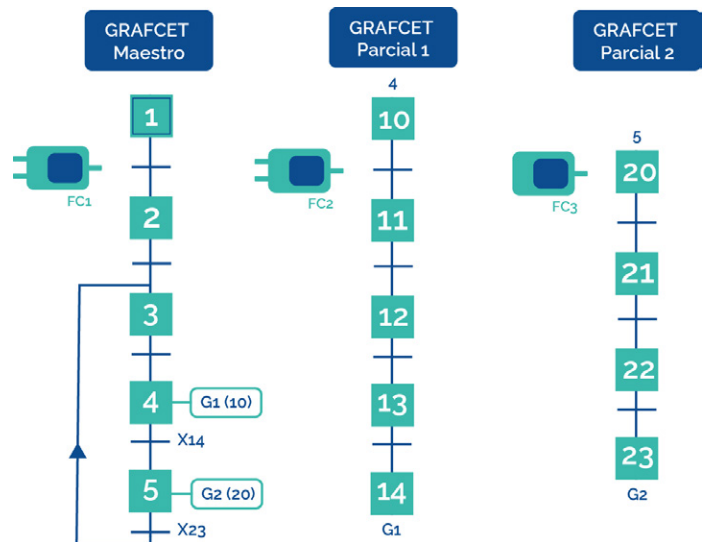


Imagen 2. Organización de la zona secuencial en varios bloques de programación

7.1.3. Zonas de acciones

Esta zona corresponderá a aquella en la cual se ejecutan las acciones en cada una de las etapas correspondientes. Lo más aconsejable es usar un único FC para programar estas zonas de acciones sobre todo para trabajar con acciones en las salidas. Sin embargo, si el programa es muy complejo podremos dividir esta zona en varios FC.



7.2.

Programación de la zona secuencial

La programación del GRAFCET se hará de una forma metódica y seguirá un patrón en el que veremos la secuencia y las acciones en cada estado.

7.2.1. Direccionamiento de las etapas

En cada etapa del GRAFCET le asociaremos un bit de la zona. Cuando vayamos a numerar el direccionamiento de dichas etapas podremos empezar en cualquier byte, pero el resto es recomendable que sean consecutivos para aportar claridad al GRAFCET.

Organización de las etapas		
Etapas	Bits de marcas	Nombres variables
Etapas 1 (inicial)	%M0.0	X1
Etapas 2	%M0.1	X2
Etapas 3	%M0.2	X3
...	...	...
Etapas 7	%M0.6	X7
Etapas 8	%M1.0	X8
...	...	...

7.2.2. Segmento modelo de etapa

En cada segmento representaremos: la etapa a la que tendremos que llegar (destino), a la que se viene (origen) y la transición que deberá flanquearse para alcanzar dicho destino.

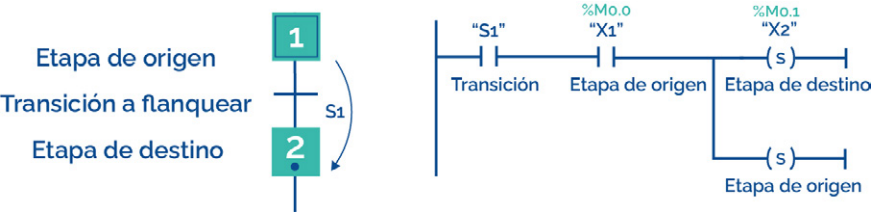


Imagen 3. Modelo de etapa en KOP

7.2.3. Combinaciones lógicas en transiciones

Esta combinación lógica de las señales que tendrá la transición se conectará en serie con dicho contacto en su etapa desde la que viene.

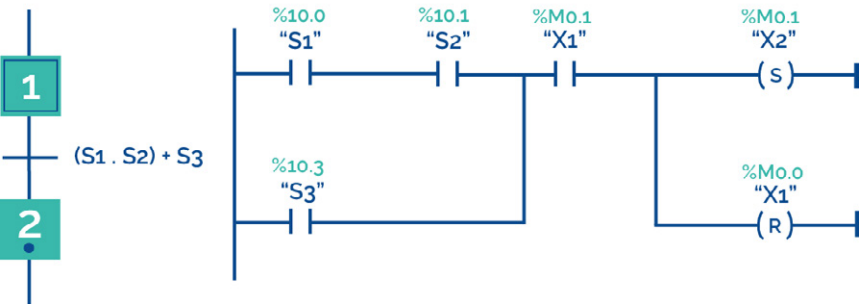


Imagen 4. Combinaciones lógicas en las transiciones

7.2.4. Divergencia de secuencias opcionales (OR)

Seguirá el mismo patrón de diseño comentado anteriormente. La programación de estas etapas de entrada será igual a lo que hemos visto para una sola etapa.

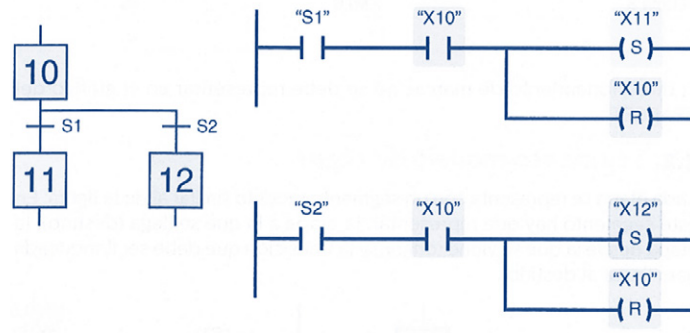


Imagen 4. Divergencia en OR

7.2.5. Convergencia de secuencias opcionales (OR)

Si queremos cerrar una secuencia con diferentes caminos opcionales será necesario poner la bobina SET en paralelo con el RESET. A dicho bloque le seguirán las combinaciones lógicas de los contactos los cuales definirán varios caminos desde los que se llegará a la etapa actual.

Estos caminos estarán formados por una red de contactos en serie, en la cual se establecerá la condición de etapa y transición desde su origen. Tendrá tantas redes en paralelos como caminos tendrá la convergencia.

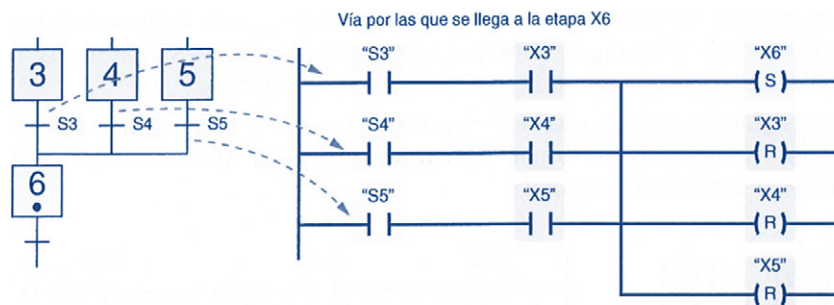


Imagen 5. Llega a una etapa desde varias secuencias opcionales

7.2.6. Divergencia de secuencias simultáneas o sincronizadas (AND)

En este tipo de divergencia, activaremos a la vez tantas etapas como ramas se vayan a ejecutar. Pondremos en el mismo segmento todas las bobinas SET en paralelo en las etapas destino y RESET en el origen.

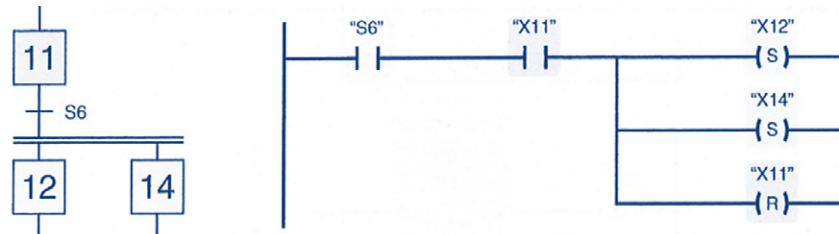


Imagen 6. Divergencia en Y

7.2.7. Convergencia de secuencias simultáneas o sincronizadas (AND)

Las secuencias simultáneas finalizarán en etapas de espera por cada uno de los caminos que se querrán sincronizar. Los bits se combinarán en serie para activar la etapa de destino. En dicha situación se desactivarán con RESET todas las etapas.

La transición =1 no tendrá ninguna señal lógica. Esto querrá decir que, si los caminos que van simultáneamente han acabado, se deberá continuar por la secuencia principal.

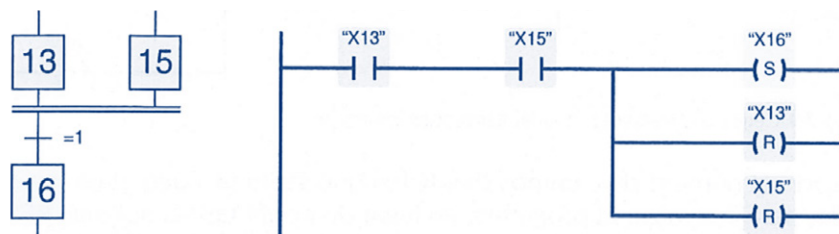


Imagen 7. Convergencia en Y

7.2.8. Temporizaciones en las transiciones

Cuando haya pasado un tiempo de permanencia podremos flaquear las transiciones. Para este propósito se usarán temporizadores a la conexión, que tendremos que activarlo con la etapa activa y la de salida se empleará en una transición como un contacto.

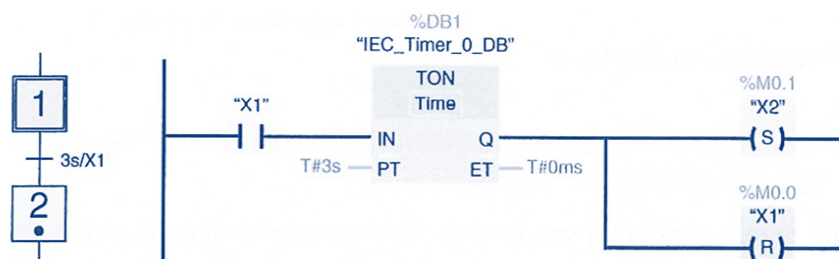


Imagen 8. Transición temporizada

7.2.9. Contadores en las transiciones

Los contadores en el GRAFCET nos permitirán direccionar el flujo de nuestra secuencia. Gestionaremos el valor del contador mediante sus entradas desde la zona de acciones.

Uso de comparaciones para direccionar el flujo de un GRAFCET

Para realizar este método necesitaremos usar una variable que almacene el valor. Se insertará la comparación de la variable con respecto al contador, en serie con la etapa de la que proviene.

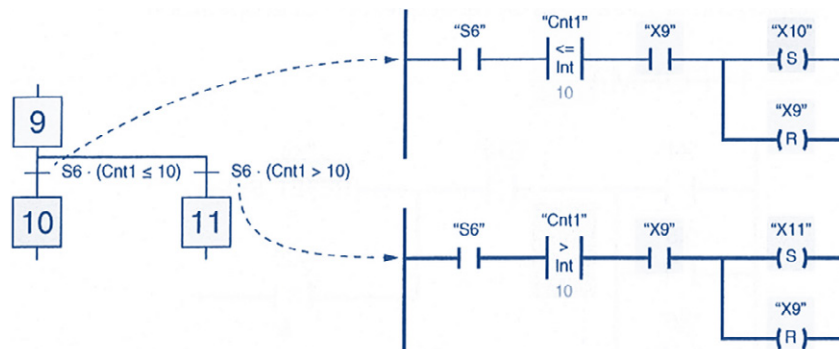


Imagen 9. Comparaciones de contadores en transiciones

Uso del bit de salida del contador

Para usarlo solo lo podremos hacer con contadores IEC, debido a que los contadores de SIMATIC no tendrán una salida booleana Q que se pueda activar cuando se alcanza un valor PV.

La programación será igual que la mencionada anteriormente, pero cambiaremos las comparaciones por contactos, ya sean abiertos o cerrados.

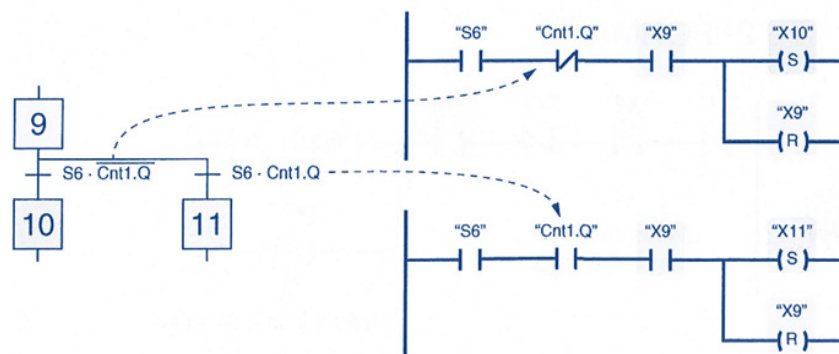


Imagen 10. Contactos de salida booleana en contadores en transiciones

7.2.10. Saltos y retornos desde múltiples etapas

En un GRAFCET los saltos y retornos permitirán direccionar el flujo del programa a una etapa. Será muy útil cuando queramos realizar otra tarea de la cadena principal y así poder abandonar la acción que estábamos haciendo cuando queramos para luego volver.

Para realizar esta operación podremos emplear dos formas:

1. Representar dichas etapas desde donde queremos hacer el salto, en este caso se tienen que poner los contactos en paralelo de los bits entre sí y el conjunto en serie con la acción del bit que ejecutará la operación.
2. Representar etapas donde no queremos hacer el salto, en este caso tendremos que poner los bits negados, con el contacto que ejecute la operación.

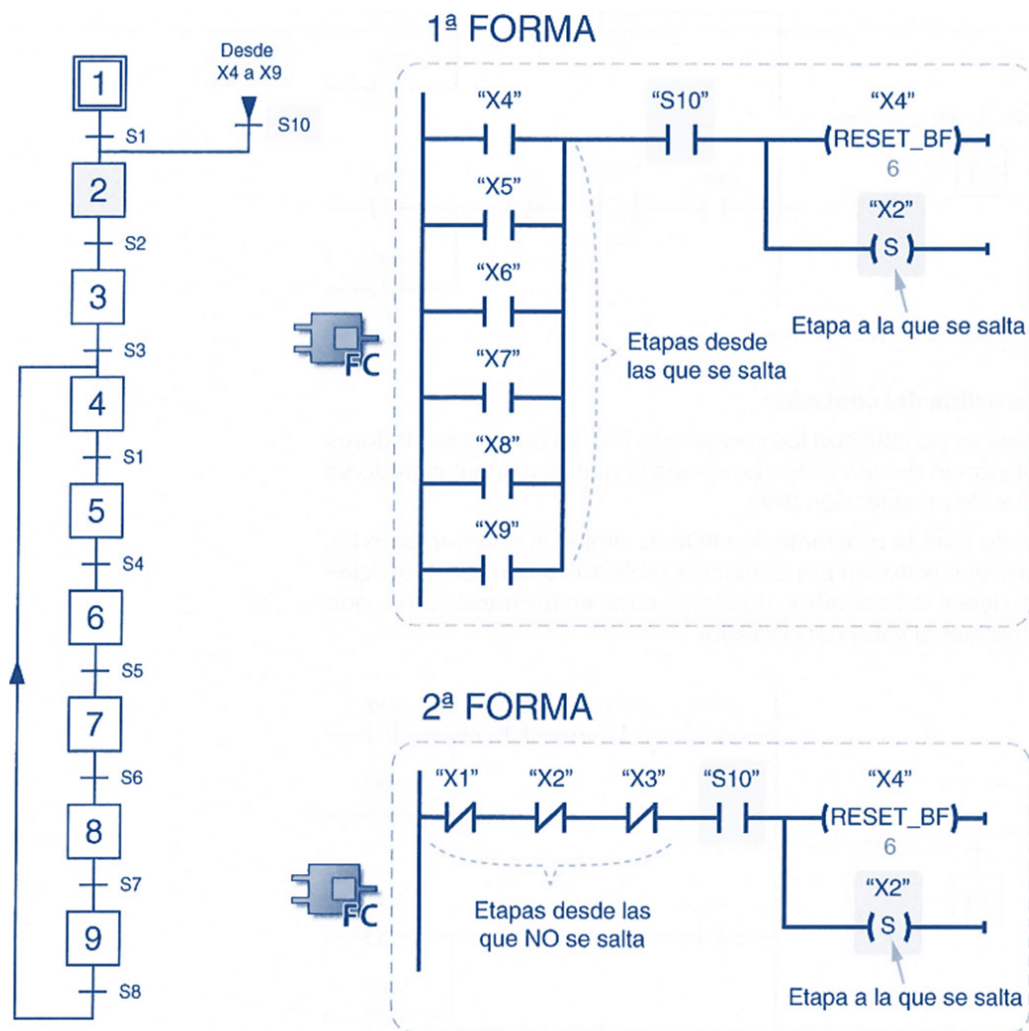


Imagen 11. Ejemplo de retorno desde múltiples etapas



7.3.

Programación de la zona de inicialización

En la zona inicial se escribirán los programas, los cuales le asignaremos a las etapas iniciales y a los datos que lo necesiten.

7.3.1. Activación de etapas iniciales

Cuando el PLC pasara de STOP a RUN, se activarán las etapas iniciales. En cualquier modelo de los PLC de Siemens lo haremos mediante un bloque de arranque OB100 (STARTUP). Aquí estarán todas las bobinas SET y RESET que queramos activar y desactivar. El FirstScan lo podremos usar en el S7-1200 Y S7-1500 para sustituir al OB100.



7.4.

Programación de la zona de acciones

7.4.1. Asignación directa o acción continua

Cuando dicha etapa este activa la acción se mantendrá y desaparece cuando la abandonamos. Como observaremos en el ejemplo hay 3 salidas que se activarán cuando la etapa 2 y 3 estén activas.

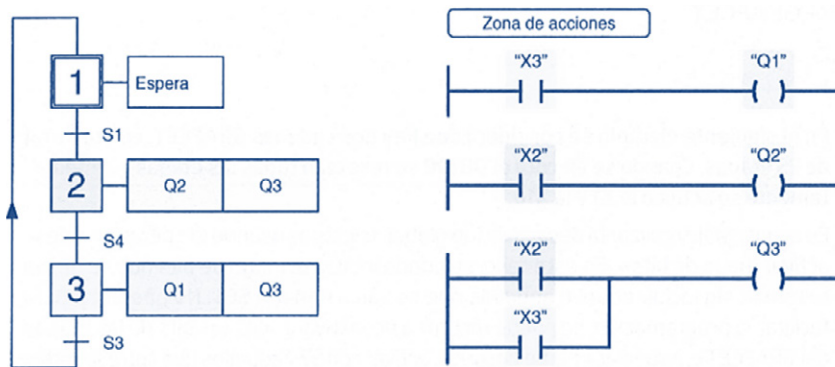


Imagen 12. Ejemplo de acciones continuas

7.4.2. Acción condicionada

Se mantendrá una acción cuando tengamos activa la etapa y además cumpla la condición de señal.

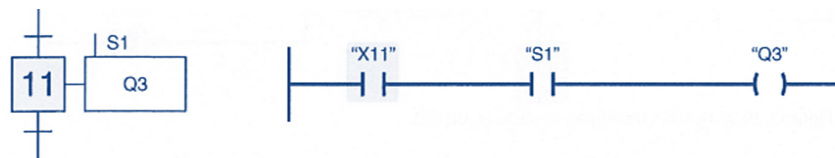


Imagen 13. Acción condicionada

7.4.3. Asignación a la activación (SET)

Pondremos un "1" lógico en una variable booleana. Y podrá a SET dicha variable.



Imagen 14. Asignación a la activación

7.4.4. Asignación a la desactivación (RESET)

Esto hará el efecto contrario al segmento comentado anteriormente, por lo que pondrá un "0" lógico.

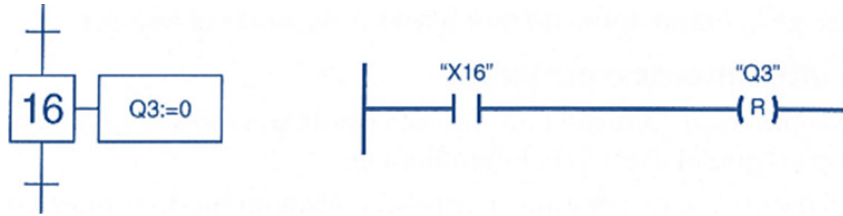


Imagen 15. Asignación a la desactivación

7.4.5. Acciones condicionadas a temporización

Acción con retardo a la conexión

Ejecutaremos la acción pasada el tiempo alcanzado en la etapa.

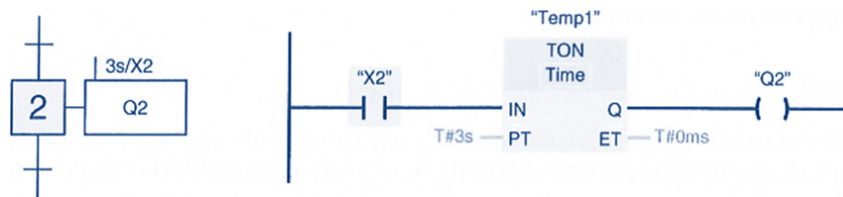


Imagen 16. Acción con retardo a la conexión

Acción límite

Aquí lo que pasará es que la acción se ejecutará cuando entra en la etapa y parará después de un tiempo límite.

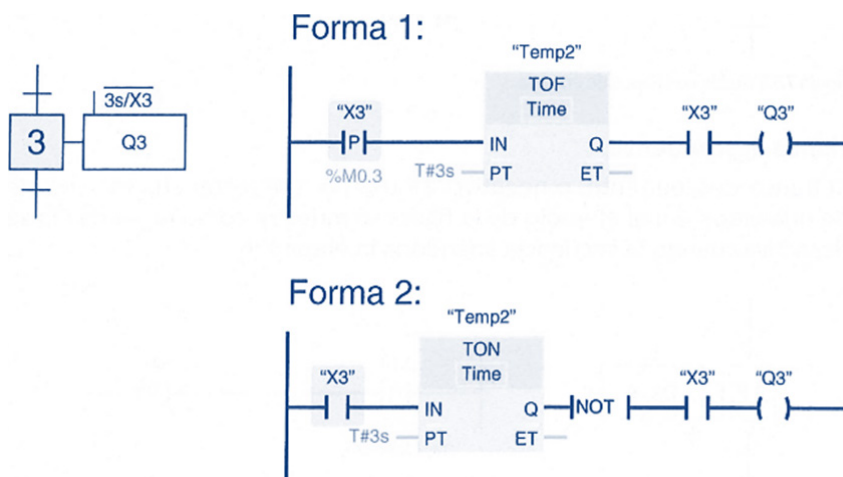


Imagen 17. Acción límite

Acción intermitente o por pulsos

Consistirá en una acción que controlará una variable booleana por pulsos simétricos. La opción más óptima será usar un bit de la marca de ciclo en serie con el bit de la etapa.

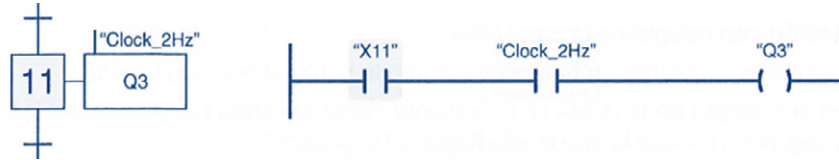


Imagen 18. Acción intermitente

7.4.6. Acciones de asignación por flanco

Flanco ascendente

Se usará para la detección de una entrada en una etapa y se le llamará flanco ascendente o positivo.

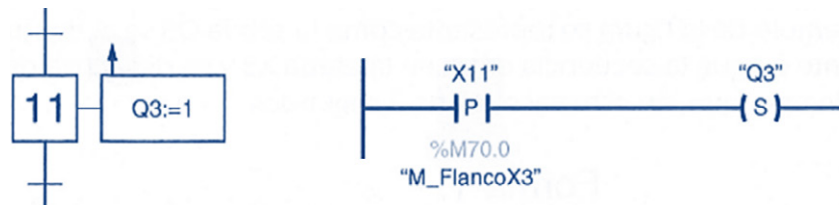


Imagen 19. Acción con flanco de entrada

Flanco descendente

Se usará para la detección de una salida en una etapa y se le llamará flanco descendente o negativo.

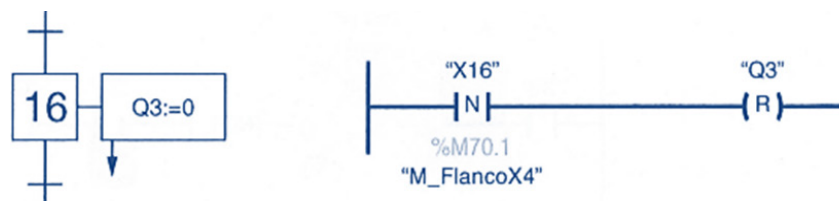


Imagen 20. Acción con flanco descendente

7.4.7. Acciones condicionadas a eventos

Si estamos en una etapa concreta estas acciones nos permitirán activar o desactivar variables booleanas, además, se activará la señal que hay al lado de la banderola.



Imagen 21. Acciones condicionadas a eventos



7.4.8. Acciones con contadores

Se usarán las entradas del bloque de un contador para poder realizar las operaciones de contar, descontar, ponerlo a RESET o ajustarlo.

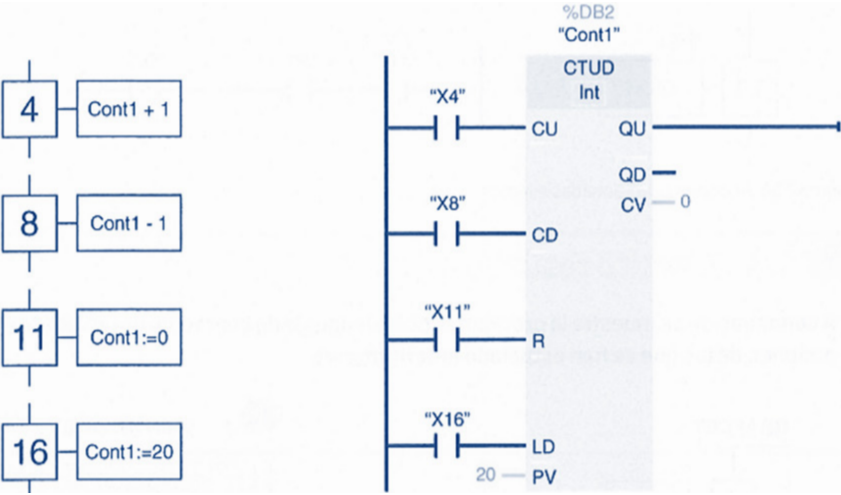


Imagen 22. Acciones sobre contador

7.4.9. Acciones en SPF

Como ya sabemos el GRAFCET estará estandarizado por la norma IEC-848 y que hay una representación gráfica para los PLC llamados SFC. En la tabla que tenemos a continuación veremos las pequeñas diferencias que hay entre ambas.

Comparativa entre las acciones del estándar IEC-848 y las del lenguaje SFC		
Acciones	Estándar	
	SFC	IEC-848
Acción directiva mientras la etapa está activa		
Activación (SET)		
Desactivación (RESET)		
Activación con retardo a la conexión		
Activación por tiempo límite		



7.5.

Zona no secuencial

En cualquier programa podremos tener una zona que no pertenecerá al GRAFCET y de tal forma que las operaciones que hagamos en ella no afectarán al funcionamiento de la secuencia. Habrá que llamarla desde un MAIN y se debe hacer en un bloque FC.

Como podemos ver en el ejemplo que tenemos a continuación veremos un detector S1 que ira incrementando los pulsos de un contador sin depender de ningún estado. Cuando este valor es mayor que 10 pasará de X9 a X10.

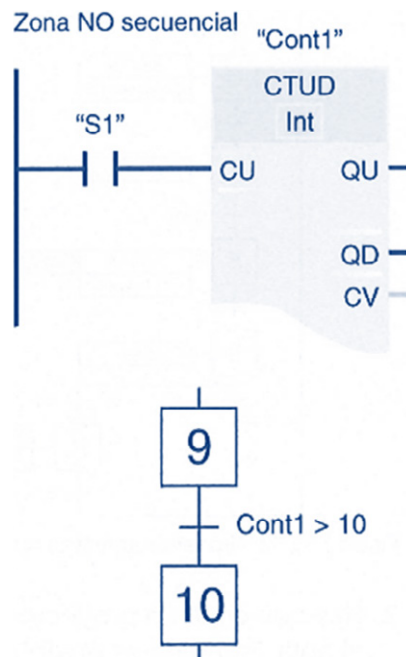


Imagen 23. Ejemplo de una zona no secuencial



 www.universae.com

